

A 404 response is a standard error response that's returned when a requested resource doesn't exist on a server. Here we import the `Http404` exception ❶, which we'll raise if the user requests a topic they shouldn't have access to. After receiving a topic request, we make sure the topic's user matches the currently logged-in user before rendering the page. If the requested topic's owner is not the same as the current user, we raise the `Http404` exception ❷, and Django returns a 404-error page.

Now if you try to view another user's topic entries, you'll see a "Page Not Found" message from Django. In Chapter 20, we'll configure the project so users will see a proper error page instead of a debugging page.

Protecting the `edit_entry` Page

The `edit_entry` pages have URLs of the form `http://localhost:8000/edit_entry/entry_id/`, where the `entry_id` is a number. Let's protect this page so no one can use the URL to gain access to someone else's entries:

```
learning_logs/
views.py      --snip--
@login_required
def edit_entry(request, entry_id):
    """Edit an existing entry."""
    entry = Entry.objects.get(id=entry_id)
    topic = entry.topic
    if topic.owner != request.user:
        raise Http404

    if request.method != 'POST':
        --snip--
```

We retrieve the entry and the topic associated with this entry. We then check whether the owner of the topic matches the currently logged-in user; if they don't match, we raise an `Http404` exception.

Associating New Topics with the Current User

Currently, the page for adding new topics is broken because it doesn't associate new topics with any particular user. If you try adding a new topic, you'll see the message `IntegrityError along with NOT NULL constraint failed: learning_logs_topic.owner_id`. Django is saying you can't create a new topic without specifying a value for the topic's owner field.

There's a straightforward fix for this problem, because we have access to the current user through the request object. Add the following code, which associates the new topic with the current user:

```
learning_logs/
views.py      --snip--
@login_required
def new_topic(request):
    --snip--
    else:
        # POST data submitted; process data.
        form = TopicForm(data=request.POST)
        if form.is_valid():
```
